Assessing Robustness and Resistance to Attacks of an Authentication System Based on OpenID Connect Protocol and Ethereum Blockchain

1st ILBOUDO Assane
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
assaneilboudo1998@gmail.com

2nd BASSOLE Didier
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
dbassole@gmail.com

3rd KOURAOGO Justin Pegdwindé
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
kouraogo@gmail.com

4rd KOALA Gouayon
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
gouayonkoala1@gmail

5rd SIE Oumarou
dept. of Computer Science
University Joseph KI-ZERBO
Ouagadougou, Burkina Faso
oumarou.sie@gmail.com

2025 15th International Conference on Electrical Engineering (ICEENG) | 979-8-3315-1901-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICEENG64546.2025.11031345

***Abstract*—In this paper, we assess the robustness and resistance against various types of attacks of a multi-factor authentication mechanism that we have proposed. It is a mechanism based on the OpenID Connect protocol and utilizes the Ethereum blockchain. Robustness was evaluated by conducting appropriate security tests using AVISPA and Scyther protocol security verification tools. The analysis of the test results confirms that our proposed authentication mechanism provides effective resistance against replay attacks, identity spoofing attacks, counterfeit token fabrication attacks, Broken End-User Authentication attacks, and man-in-the-middle attacks. We also evaluated performance in terms of gas cost in Ether consumed during different transactions as well as their execution times.**

***Index Terms*—Single Sign-On, Robustness, Ethereum Blockchain, OpenID Connect**

I. Introduction

Authentication is a fundamental property in computer security. It enables a system to verify the legitimacy of an access request made by an entity (human or another system...) in order to authorize that entity's access to system resources (systems, networks, applications...) according to the access control settings. Confidentiality, integrity, and non-repudiation cannot be ensured without guaranteeing the identity of the entity submitting a request. Among the types of authentication, we have single sign-on and multi-factor authentication. Multi-factor authentication (MFA) is an authentication process where the user must present at least two distinct verification methods to access a resource such as an application, an online account, or a virtual private network (VPN). MFA is a fundamental element of any robust identity and access management strategy. Instead of relying solely on a username and password, MFA requires the use of one or more additional verification factors, thereby significantly reducing the chances of a successful cyberattack. Ensuring the identity of an entity to guarantee other security properties is a priority for cybersecurity actors, but having robust authentication systems resistant to cyberattacks is a major concern for the security research community. In this context, evaluating the robustness of single sign-on authentication systems against attacks is a major concern for researchers in the field. The objective of this work is to evaluate the robustness and resistance against various types of cyber attacks of a multi-factor authentication mechanism that we have proposed. Our system has a secure architecture integrating a decentralized multi-factor authentication mechanism based on the Ethereum blockchain and using the OpenID Connect protocol. The remainder of this paper is organized as follows: Section II focuses on the related works, Section III on architecture of our Ethereum-integrated authentication approach. In Section IV, we analyze the security of our authentication mechanism using the AVISPA and Scyther tools. Section V focuses on evaluating the robustness and resistance of the proposed mechanism against various types of attacks. Section VI is dedicated to evaluating the performance of our authentication mechanism in terms of gas cost in Ether consumed during different transactions as well as their execution times. We conclude this work in Section VII.

II. Related Work

To ensure the confidentiality and privacy of online users, numerous studies have been conducted in the field of authentication systems, leveraging the advantages of blockchain combined with single sign-on to address security challenges in authentication systems. Tanriverdi [1] proposed a secure authentication system based on blockchain to address the limitations of traditional authentication systems, LDAP, and third-party applications like Google. These systems are vulnerable to brute force attacks, credential reuse, and dependency on third-

Authorized licensed use limited to: Nokia. Downloaded on July 22,2025 at 12:47:25 UTC from IEEE Xplore. Restrictions apply.

party systems. Mir et al. [2] proposed a decentralized privacy-preserving authentication system (DMFA) that eliminates the need for sensitive user data for identity providers and prevents tracking of individual user activity. The protocol allows service providers to authenticate users without relying on a centralized identity provider. Zhanfei et al. [3] proposed a secure and distributed authentication scheme based on blockchain for the Internet of Vehicles (IoV). Their approach aims to overcome the limitations of traditional authentication systems, which rely primarily on Road Side Units (RSUs). Indeed, these centralized systems present several major drawbacks, including RSU overload, increased latency due to the need to synchronize updates across multiple units, and overall inefficiency in the authentication process. Moreover, the absence of an effective mechanism for reusing authentication results leads to excessive network resource consumption and increased computational costs. It is in this context that Lim et al. [4] proposed a blockchain-based authentication system called AuthChain. In this system, each user generates a pair of public and private keys for an account on a trusted device. The public key serves to identify each user, securely stored on the user's device (in secure Android storage or iOS keychain) and providing passwordless access. Yousra et al. [5] proposed a new model to enhance the security of OpenID Connect using the Ethereum Blockchain standard and non-fungible tokens (ERC721). The goal is to prevent attacks aiming to impersonate legitimate entities, gain unauthorized access, take control of user accounts, or violate their privacy. However, it should be noted that despite the improvements made, the system presented remains vulnerable to potential threats concerning IdP (identity provider) accounts. Danda et al. [6] proposed a decentralized authentication framework using decentralized identifiers (DIDs) and secret sharing to enhance security and privacy in the OAuth2.0 protocol.

We will direct our research towards the OpenID Connect protocol, particularly towards Yousra's work aiming to enhance its security. However, user account privacy is compromised, making the system vulnerable to phishing attacks. We plan to enhance this privacy by integrating a blockchain-based second-factor authentication mechanism. We have chosen this protocol for several reasons. Firstly, widely used single sign-on authentication protocols today include OpenID Connect, supported by major digital companies such as Microsoft, Amazon, Google, Meta, PayPal, Verizon, Salesforce, Oracle, VMware, IBM, WordPress, Yahoo, GitHub, and Twitter. Additionally, OpenID Connect is considered the new standard in single sign-on authentication systems [7] [8] [9].

III. Architecture of the Authentication Mechanism Integrating Ethereum

Our multi-factor authentication mechanism based on OpenID Connect and integrating the Ethereum blockchain is illustrated in the following Figure 1.

This authentication system based on OpenID Connect and utilizing the Ethereum blockchain primarily involves three (03) actors: the User, the Identity Provider (IdP), the Relying Party (RP).

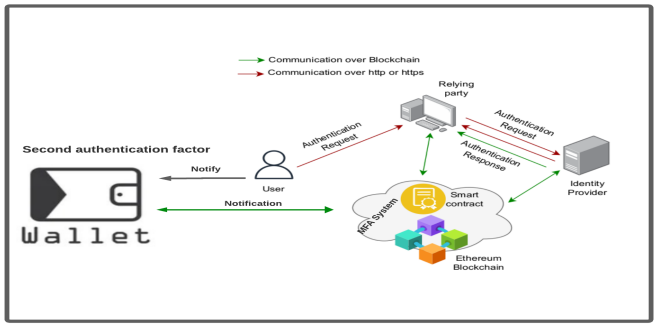

Fig. 1. Description of our methodological approach

To ensure the security of the process, we used the *_safeMint()* method to create a new token, associating the tokenId with the jti attribute of the JWT and the _data with the byte32 code of the JWT. Additionally, we designated the address of the contract owner (the IdP) as the destination address. Next, we transferred the token from the IdP to the RP on the Ethereum network using the *transferFrom()* method. We also used the *approve()*, *ownerOf()*, and *tokenURI()* functions to respectively authorize token transfer, verify the token owner, and obtain the token's metadata URI. Regarding the user authentication process, we use the *mfa()* method within our smart contract to implement a decentralized multifactor authentication system. This method aims to send a confirmation transaction to the user's Metamask wallet, thereby soliciting their validation. Confirmation of this transaction is equivalent to a demonstration of identity by the user to the trusted third party, represented by the smart contract.

We describe our solution aimed at enhancing security and ensuring the confidentiality of user data within the OpenID Connect protocol environment by integrating blockchain technology and establishing a multi-factor authentication mechanism. This approach aims to guarantee the integrity, availability, and confidentiality of the OpenID Connect security parameters while restricting access to authorized entities through a robust verification process. Additionally, we improve the security of user authentication with their identity provider by introducing a multi-factor authentication mechanism. Our system model is depicted in Figure 2, detailing the interactions between stakeholders. Our architecture is divided into four (04) phases:

- *The registration of the RP with the IdP*;
- *The registration of the user with the IdP*;
- *The authentication of the user with the IdP*;
- *The authentication of the user with the RP*.

In Table I, we present descriptions of various notations used within our system architecture.

IV. Implementation and Security Analysis

We have developed a smart contract with two main functionalities. The first aims to verify the legitimacy of the service

Authorized licensed use limited to: Nokia. Downloaded on July 22,2025 at 12:47:25 UTC from IEEE Xplore. Restrictions apply.

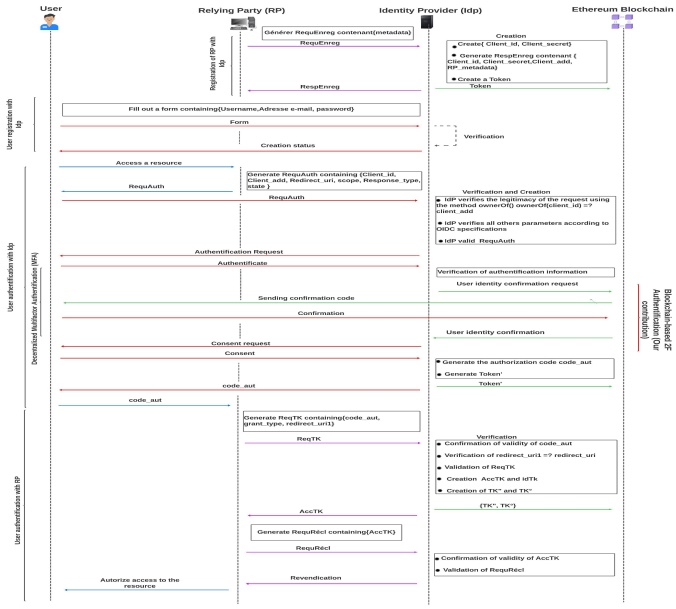

Fig. 2. Description of the interactions between stakeholders

TABLE I
DESCRIPTION OF THE VARIOUS NOTATIONS USED IN OUR SYSTEM ARCHITECTURE

Notation	Description
AccTK	Access token
aud	Public audience of this token
Client_add	Required parameter in our system with the value being the blockchain address of the RP
Client_id	Unique RP identifier
Client_secret	Secret value used by clients to authenticate with the IdP
code_aut	Authorization code
exp	Token expiration time
idTk	Identification token containing user claims
iss	Token issuer
jti	Unique token identifier
Redirect_uri	Redirect URI where the response will be sent
ReqRécl	Claim request
ReqTK	Token request
RequAuth	Authentication request
RequEnreg	RP registration request
RespEnreg	Registration response
response_type	Required parameter where we specify the flow being used
RP_metadata	RP metadata
scope	Required parameter in OIDC where we specify the set of claims needed to authenticate an end-user
state	A recommended parameter used to mitigate Cross-Site Request Forgery (CSRF) attacks
sub	Token subject

provider, while the second consists of a dual authentication mechanism strengthening traditional authentication (username and password). This smart contract has been deployed on the Ganache[1] blockchain. For our tests, including the smart contract and the OpenID Connect protocol, we integrated the ABI of our smart contract, the IdP address, and a JSON Web Token (JWT) to evaluate the transfer of OIDC security parameters on the Ethereum network.

A. Implementation tools

Before evaluating the robustness of our system, we implemented it using the following tools:

- **Ethereum Remix**[2];
- **AVISPA**[3](Automated Validation of Internet Security Protocols and Applications);
- **Scyther**[4] is a cybersecurity tool designed for the formal analysis and verification of security protocols;
- **Postman**[5];
- **MetaMask**[6] is a cryptocurrency software wallet used to interact with the Ethereum blockchain;
- **HLPSL**[7] (High-Level Protocol Specification Language) is a high-level specification language used to formally describe and model communication protocols and distributed systems;
- **Solidity**[8] is a programming language specifically designed for developing smart contracts on the Ethereum blockchain.

B. Security Analysis with AVISPA and Scyther

We conducted a simulation of our authentication system and utilized the AVISPA and Scyther tools to perform a formal security analysis and verification. The principles of AVISPA and Scyther are based on the *Dolev-Yao* adversary model, and they have been employed by other authors [5] [10] [11] [12] [13] [14] for the formal verification of protocols. To assess the security, particularly the confidentiality and strong/weak authentication, of our system with AVISPA, we described it using the HLPSL (High-Level Protocol Specification Language).

- Two (02) objectives of user authentication with the IdP:
  1) The first is to ensure that the user authentication process with the IdP is robust.
  2) The second is to include criteria such as verifying the user's identity, protecting against identity spoofing attacks, and validating the legitimacy of the authentication request.
- Two (02) objectives of token confidentiality, including:
  1) The first aims to ensure that the information contained in the tokens generated by the system is secured against unauthorized access.
  2) The second encompasses measures such as encrypting sensitive data, protection against attacks aiming to compromise token confidentiality, and ensuring secure transmission of these tokens between concerned parties. Summaries of the OFMC and CL-AtSe simulations in Figures 3 and 4 confirm the

[1]https://archive.trufflesuite.com/ganache/

[2]https://remix-project.org
[3]https://www.avispa-project.org
[4]https://people.cispa.io/cas.cremers/scyther/install-generic.html
[5]https://www.postman.com
[6]https://metamask.io
[7]https://www.semanticscholar.org/paper/The-High-Level-Protocol-Specification-Language-in-Oheimb
[8]https://docs.soliditylang.org/

Authorized licensed use limited to: Nokia. Downloaded on July 22,2025 at 12:47:25 UTC from IEEE Xplore. Restrictions apply.

safety of our protocol. The translation time in CL-AtSe is 0.01 seconds, and the search time in OFMC is 0.02 seconds to visit 16 nodes with a depth of 4 stacks.

2

```
% OFMC
% Version of 2006/02/13
SUMMARY
  SAFE
DETAILS
  BOUNDED_NUMBER_OF_SESSIONS
PROTOCOL
  /home/span/span/testsuite/results/BlockchainOIDC.if
GOAL
  as_specified
BACKEND
  OFMC
COMMENTS
STATISTICS
  parseTime: 0.00s
  searchTime: 0.02s
  visitedNodes: 16 nodes
  depth: 4 plies
```

Fig. 3. AVISPA verification result with the OFMC backend

```
SUMMARY
  SAFE

DETAILS
  BOUNDED_NUMBER_OF_SESSIONS
  TYPED_MODEL

PROTOCOL
  /home/span/span/testsuite/results/BlockchainOIDC.if

GOAL
  As Specified

BACKEND
  CL-AtSe

STATISTICS

  Analysed    : 4 states
  Reachable   : 0 states
  Translation: 0.01 seconds
  Computation: 0.00 seconds
```

Fig. 4. AVISPA verification result using the CL-AtSe backend

In addition to the analysis conducted with AVISPA, we also reexamined the security of our proposed system using the Scyther tool. Indeed, Scyther offers additional features that are not available in AVISPA. Scyther ensures the verification of protocol security. This applies to a number of sessions, whether bounded or unbounded. It offers multi-protocol analysis, which checks the concatenation of their description files. Scyther generates attack graphs when it detects vulnerabilities. Scyther requires as input a formal description of the security protocol to be analyzed, which is specified in the form of a file in the Standard Page Description Language (SPDL), a language used for describing security protocols. In the SPDL file of our OpenID Connect system based on the Ethereum blockchain, we defined the same three (03) roles, namely the *User*, the *RP*, and the *IdP*. Within these roles, we established claims aimed at verifying confidentiality (Secrecy) as well as other authentication properties such as liveness (Alive), non-injective synchronization (Nisynch), non-injective agreement (Niagree), and weak agreement (Weakagree). Figure 5 illustrates the results of our analysis and evaluation. The verification conducted with Scyther shows that no successful attacks were identified against our system.

We explain in the table II the properties used during our simulation, illustrated in the figure 5

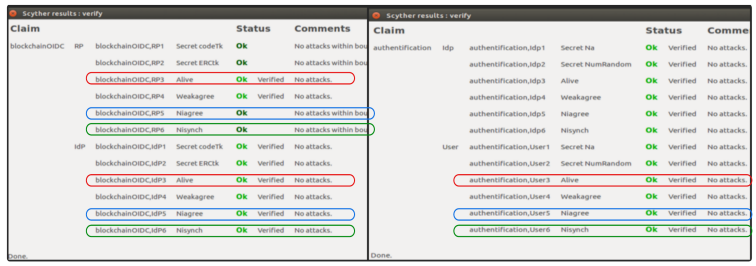

Fig. 5. Results of the evaluation of the proposed model by Scyther

TABLE II
EXPLANATION OF OTHER AUTHENTICATION PROPERTIES

Other authentication properties	Explication
Alive	This property ensures that an authentication protocol does not remain indefinitely blocked. In other words, it guarantees that participants in the protocol will eventually advance the authentication process
Nisynch	Non-injective synchronization means that each message from a participant must be responded to by at most one other message. This prevents interference from unwanted elements or malicious manipulation of messages.
Niagree	Non-injective agreement means that a participant cannot force another participant to accept a message that they did not originally send. It prevents falsification of agreements between participants.
Weakagree	Weak agreement is an authentication property that states if one participant accepts a message, then other participants will also accept that message. This reinforces the idea that message acceptance is consistent across participants.

V. EVALUATION OF THE ROBUSTNESS OF THE PROPOSED AUTHENTICATION MECHANISM

A. Evaluation of the system's resilience against various attacks

Resistance to replay attacks (Highlighted in the figure 5 within a red border): A replay attack allows an attacker to intercept and record legitimate data, such as tokens, to replay them later and gain unauthorized access to restricted resources. If the attacker manages to acquire any of the security parameters of OpenID Connect, such as the access token, authorization code, redirect_uri, and client_id, our system based on the Ethereum blockchain will ensure authenticity. It verifies not only the parameters included in a request but also the identity of the issuer through the *OwnerOf()* function of the smart contract. For example, if the attacker tries to reuse a legitimate access token to obtain user authentication or an id_token, the system will reject this attempt because the *ownerOf()* output address of this token will not match that of the attacker.

Resistance to RP Identity Spoofing Attacks (Highlighted in the figure 5 within a green border): An attacker may impersonate legitimate entities and access protected resources by obtaining sensitive parameters from OpenID Connect trans-

Authorized licensed use limited to: Nokia. Downloaded on July 22,2025 at 12:47:25 UTC from IEEE Xplore. Restrictions apply.

mitted during authentication flows. Our authentication mechanism prevents this attack through our rigorous verification process. If the attacker attempts to impersonate the user and sends a request to access restricted resources using a recently fraudulently acquired token, their request will be rejected because the attacker's Ethereum address does not match either the one contained in the "sub" attribute of the JWT or the output address of the *ownerOf()* function.

Resistance to Fake Token Fabrication Attacks (Highlighted in the figure 5 within a blue border): A strategy where fake tokens mimic authentic ones allows a malicious individual to distribute them to illicitly gain entry into RP resources and access users' confidential information. Our system thwarts this maneuver by creating an ERC-721 token for each token generated by the IdP, which is transmitted via the blockchain network. This allows the RP to confirm if the token comes from the legitimate address of the IdP. It simply verifies the public ledger. Additionally, the system checks if the string returned by the *tokenURI()* function matches the base64url representation of the received token.

Revocation and Delegation: In our proposal, revocation of an RP's access is done through the *transferFrom()* function of the ERC-721 contract, thus returning the token to the IdP. If the RP has already used the JWT and the token is associated with a session identifier, any subsequent attempt by the RP to use this token after its revocation will fail. This occurs because the output of the *ownerOf()* function will no longer match the EthRP Ethereum address of the RP indicated in the "sub" attribute of the JWT. Furthermore, our system ensures the delegation function, allowing the user to log into their RP account from a device that does not have access to the EthRP address and delegate tokens to another Ethereum address using the *approve()* function.

Resistance to Broken End-User Authentication Attacks: A "Broken End-User Authentication" attack refers to any situation where the end-user authentication process is compromised, altered, or faulty. This can occur in various ways, such as vulnerabilities in authentication protocols, ineffective security practices, defects in software implementations, or human errors. Our system prevents this attack through a decentralized multi-factor authentication mechanism based on the Ethereum blockchain. The IdP uses the smart contract on the Ethereum blockchain to initiate the generation of a random number distributed to both the IdP and the user. This random number becomes a second authentication factor, complementing the user's credentials and attesting to their authorization to access the account. This double authentication enhances the security of the process: even if an attacker manages to obtain the user's credentials, access is still denied without this random number.

B. Summaries and Comparative Study

We conducted a comparative study of our Ethereum blockchain-based authentication mechanism model with similar works mentioned in the literature review. Our model withstands attacks such as replay attacks, identity spoofing attacks, fake token fabrication attacks, and Broken End-User Authentication attacks. Additionally, it ensures access revocation and delegation. In table III, our model is compared to other similar works. The indicators used in table III are:

- X indicates that the model does not provide the corresponding property;
- V indicates that the model provides the corresponding property;
- NA indicates that the model does not have information on the corresponding property.

TABLE III
COMPARISON OF THE PROPOSED MODEL WITH OTHER EXISTING MODELS

	OIDC	Tanriverdi [1]	Zhanfei [3]	Lim [4]	Yousra [5]	Our model
Resistance to replay attacks	X	V	NA	NA	V	V
Resistance to token forgery attacks	X	V	NA	NA	V	V
Resistance to RP identity spoofing attacks	X	V	NA	NA	V	V
Revocation and delegation	X	V	V	V	V	V
Resistance to broken end-user authentication attacks	X	X	X	NA	X	V

From this comparative study, we can affirm that our authentication mechanism proves to be robust and resistant to the various attacks listed in table III.

VI. PERFORMANCE EVALUATION

The proposed multi-factor authentication mechanism requires the use of smart contract functions, resulting in fees due to gas consumption. Gas is the fee required to ensure the success of transactions on Ethereum, and it is expressed in gwei, a subunit of ETH. The cost in Ether can be calculated as follows: $(\mathbf{GasCost} \times \mathbf{GasPricePerUnit}) \times \mathbf{10}^{-9}$ According to [15], the average Ethereum gas price on Mars 24, 2025, was 2.091 gwei. We can now evaluate the performance of our system in terms of time. It is essential to note that transaction speed, i.e., the number of transactions processed per second within a blockchain network, depends on various factors such as block time, gas limit, and transaction gas cost. These factors refer respectively to the time between mining of new blocks and transactions (expressed in seconds), the maximum amount of gas available for each transaction, and the gas fees required to perform a transaction or execute a contract on the blockchain. In accordance with [16], transaction speed is calculated using the following formula:

$$\mathbf{TransactionSpeed} = \frac{\mathbf{BlockGasLimit}}{\mathbf{TransactionGasCost} \times \mathbf{BlockTime}}$$

Authorized licensed use limited to: Nokia. Downloaded on July 22,2025 at 12:47:25 UTC from IEEE Xplore. Restrictions apply.

To ensure the reliability of the results, we set the block gas limit and block time to their current values as of Mars 24, 2025. According to [17], the block time on Ethereum is 12.04 seconds, while the block gas limit is 35,953,397 gas units. Table IV presents the gas costs and associated times for each operation of our system, ranging from deploying the contract on the Ethereum Ganache network to confirmation code. It is important to note that the tokenURI and ownerOf functions are view functions, thus incurring no cost or computational effort, as their sole function is to read the current state of the blockchain.

TABLE IV
COST OF CONSTRUCTION ELEMENTS

Operation	Cost measured in gas	Gas cost in Ether	Transaction speed (ms)
Contract deployment	1502587	0,0031419	1,987347516
Create a token	25178	0,0000526	118,602055029
Burn a token	25100	0,0000524841	118,970619184
Transfer a token	30083	0,000062904	99,264120650
Approve	24483	0,000051194	121,968816792
Confirmation transaction	43609	0,000091186	68,475831629

Table IV provides a detailed breakdown of the time required for each operation performed by the smart contract within our system. Based on these results, we can conclude that integrating the smart contract into our authentication system based on the OpenID Connect protocol will not affect its performance.

VII. CONCLUSION

In our approach, our major innovation lies in the integration of a decentralized multi-factor authentication mechanism within OpenID Connect. This advancement ensures enhanced protection of user accounts against potential threats from the identity provider. The detailed description of our system architecture, along with the presentation of the tools used, provides a comprehensive view of the implementation of our solution. Performance evaluation, conducted through the deployment of our smart contract on a local blockchain via Ganache, as well as security test analyses using tools such as AVISPA and Scyther, has rigorously demonstrated the robustness and effectiveness of our approach. The comparison with other systems listed in the literature review has fueled an in-depth discussion. Future perspectives are primarily focused on integrating an artificial intelligence model into our system to proactively analyze user behaviors, thereby detecting anomalies such as unusual login attempts. Additionally, we plan to incorporate an audit mechanism, which is essential for proactive management of security incidents, regular authorization reviews, and continuous security improvement. The integration with authorization review enhances security by preventing risks associated with inappropriate permissions. Furthermore, the audit mechanism, through regular log analysis, promotes a proactive evolution of security controls, thereby ensuring a robust posture over time.

REFERENCES

[1] Tanrıverdi, M. (2023). "Blockchain-Based Secure Authentication Solution for Web Applications". Gazi Mühendislik Bilimleri Dergisi, 9(3), 477-489.https://dergipark.org.tr/en/pub/gmbd/issue/82372/1377956

[2] Mir, O., Roland, M., & Mayrhofer, R. (2022). "Decentralized, Privacy-Preserving, Single Sign-On". Security and Communication Networks, 2022(1), 9983995.https://onlinelibrary.wiley.com/doi/abs/10.1155/2022/9983995

[3] Ma, Z., Jiang, J., Wei, H., Wang, B., Luo, W., Luo, H., & Liu, D. (2024). A blockchain-based secure distributed authentication scheme for internet of vehicles. IEEE Access. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10547210

[4] Lim, S. Y., Fotsing, P. T., Musa, O., & Almasri, A. (2023). "AuthChain: a decentralized blockchain-based authentication system". International Journal of Engineering Trends and Technology (IJETT), 70-74.

[5] Yousra, B., Yassine, S., Yassine, M., Said, S., Lo'ai, T., & Salah, K. (2023). A novel secure and privacy-preserving model for OpenID connect based on blockchain. IEEE Access. DOI : 10.1109/ACCESS.2023.3292143 https://ieeexplore.ieee.org/abstract/document/10172179/

[6] Krishna, D. P., Ramaguru, R., Praveen, K., Sethumadhavan, M., Ravichandran, K. S., Krishankumar, R., & Gandomi, A. H. (2023). "SSH-DAuth: secret sharing based decentralized OAuth using decentralized identifier". Scientific Reports, 13(1), 18335. DOI: https://doi.org/10.1038/s41598-023-44586-6. https://www.nature.com/articles/s41598-023-44586-6

[7] Jannett, L., Mainka, C., Westers, M., Mayer, A., Wich, T., & Mladenov, V. (2024, July). SoK: SSO-MONITOR-The Current State and Future Research Directions in Single Sign-on Security Measurements. In 2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P) (pp. 173-192). IEEE. https://nerd2.nrw/wp-content/uploads/2024/05/sso-monitor.pdf

[8] Innocenti, T., Jannett, L., Mainka, C., Mladenov, V., & Kirda, E. (2024, November). "Only as Strong as the Weakest Link": On the Security of Brokered Single Sign-On on the Web. In 2025 IEEE Symposium on Security and Privacy (SP) (pp. 24-24). IEEE Computer Society.https://innotommy.com/Identity_Brokers.pdf

[9] Malekolkalami, M., & Hassanzadeh, M. (2024). Digital Transformation in Libraries: Identifying the Main Components in a Knowledge Extraction System Based on Single Sign On. International Journal of Metaverse & Virtual Transformation, 1(1), 19-28.https://mvt.artahub.ir/article_193077_48e6017e9de6e6ea44d9a249c4e31c83.pdf

[10] Rakotonirina, I., Barthe, G., & Schneidewind, C. (2024). Decision and complexity of Dolev-Yao hyperproperties. Proceedings of the ACM on Programming Languages, 8(POPL), 1913-1944.https://dl.acm.org/doi/pdf/10.1145/3632906

[11] Ammann, M., Hirschi, L., & Kremer, S. (2024, May). DY fuzzing: formal Dolev-Yao models meet cryptographic protocol fuzz testing. In 2024 IEEE Symposium on Security and Privacy (SP) (pp. 1481-1499). IEEE. https://eprint.iacr.org/2023/057.pdf

[12] Rakotonirina, I., Barthe, G., & Schneidewind, C. (2024, January). Decision and complexity of Dolev-Yao hyperproperties (technical report). In Symposium on Principles of Programming Languages (POPL). https://hal.science/hal-04261390v2/file/main.pdf

[13] Sivaraman, N., Nadjm-Tehrani, S., & Johansson, T. (2024, August). Formal Analysis of Julia Key Agreement Protocol. In International Conference on Information and Communications Security (pp. 171-190). Singapore: Springer Nature Singapore. https://www.ida.liu.se/labs/rtslab/publications/2024/navya24.pdf

[14] Sivaraman, N., Nadjm-Tehrani, S., & Johansson, T. (2024, August). Formal Analysis of Julia Key Agreement Protocol. In International Conference on Information and Communications Security (pp. 171-190). Singapore: Springer Nature Singapore. https://www.ida.liu.se/labs/rtslab/publications/2024/navya24.pdf

[15] Yadav, A. K., Singh, K., Amin, A. H., Almutairi, L., Alsenani, T. R., & Ahmadian, A. (2023). "A comparative study on consensus mechanism with security threats and future scopes: Blockchain". Computer Communications, 201, 102-115.https://www.sciencedirect.com/science/article/pii/S0140366423000269

[16] Ta, M. T., & Do, T. Q. (2024). A study on gas cost of ethereum smart contracts and performance of blockchain on simulation tool. Peer-to-Peer Networking and Applications, 17(1), 200-212.

[17] Etherscan, Ethereum average block time chart. (2025).https://etherscan.io/chart/blocktime

Authorized licensed use limited to: Nokia. Downloaded on July 22,2025 at 12:47:25 UTC from IEEE Xplore. Restrictions apply.